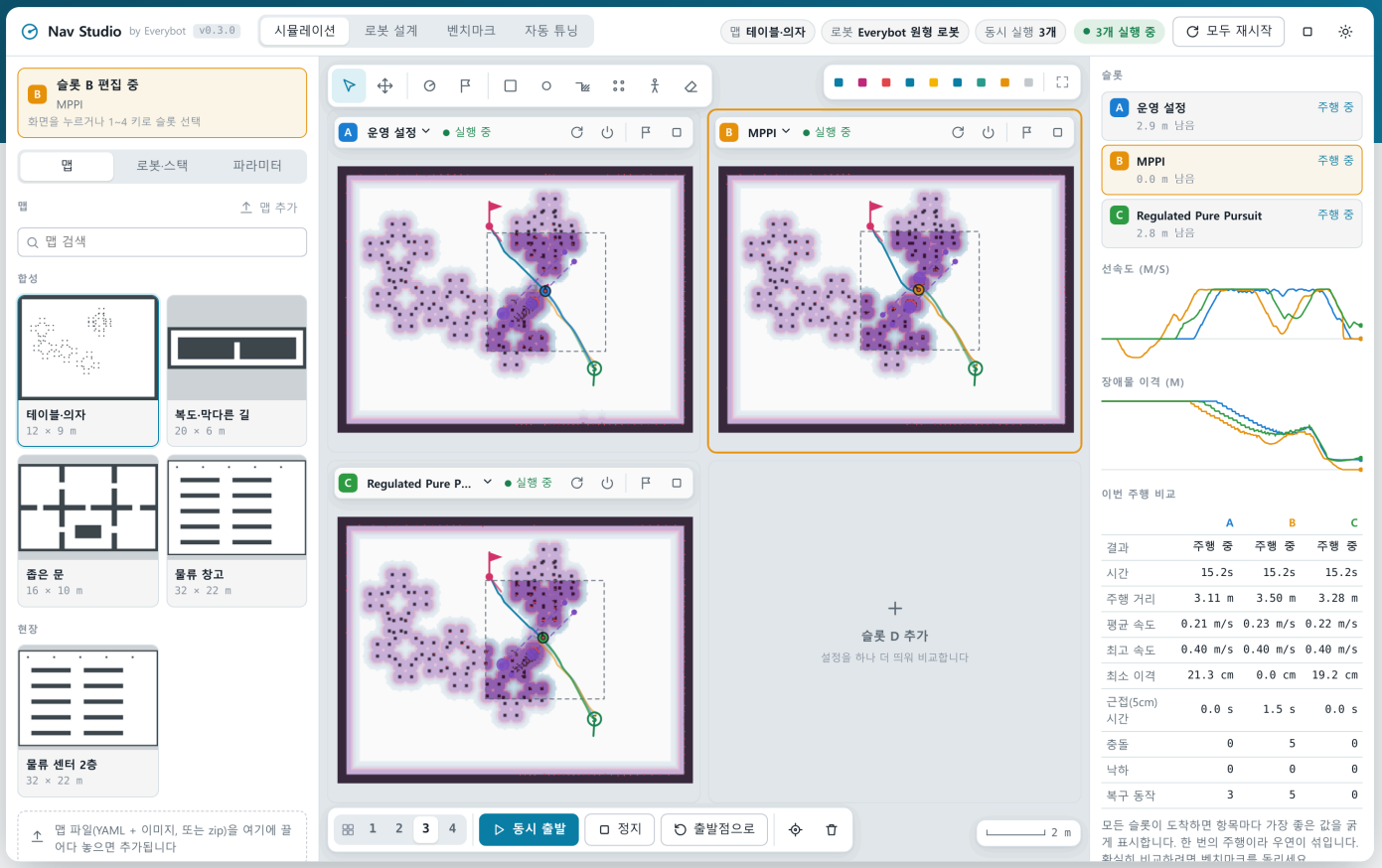
Everybot

Nav Studio

사용 가이드

브라우저에서 실제 Nav2 스택으로 로봇 주행을 시뮬레이션하고, 로봇과 센서를 설계하고, 설정 여러 개를 나란히 띄워 비교하고, 벤치마크와 자동 튜닝으로 검증합니다.

버전 0.3.0 · 2026-10-23



시뮬레이션

맵·장애물·보행자를 놓고 실시간 주행

동시 실행 비교

설정 2~4개를 나란히, 같은 순간에 출발

로봇 · 센서 설계

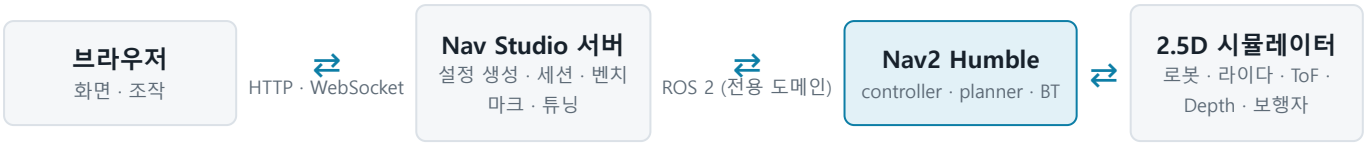
외곽·라이다·ToF·Depth 를 끌어서 배치

벤치마크 · 튜닝

통계로 비교하고 Optuna 로 탐색

Nav Studio 는 무엇인가

Nav Studio 는 Everybot 로봇의 주행 설정(Nav2 Humble)을 PC 에서 바로 시험하는 앱입니다. 브라우저 화면의 로봇은 실제 Nav2 노드 (controller · planner · behavior tree · costmap · velocity smoother)가 움직이고, 로봇과 센서는 2.5D 시뮬레이터가 대신합니다. 실제 로봇 없이 파라미터를 바꿔 보고, 바꾼 효과를 숫자로 비교하고, 더 나은 값을 찾을 수 있습니다.



작업 공간

시뮬레이션	맵을 고르고 목표를 찍으면 로봇이 주행합니다. 박스·기둥·의자·보행자를 실행 중에 놓을 수 있고, costmap·경로·라이다·속도·장애물 이격거리를 실시간으로 봅니다. 파라미터를 바꾸면 Nav2 를 다시 띄워 적용합니다. **동시 실행**으로 설정이 다른 시뮬레이션을 최대 4개 나란히 띄워 비교하고, 스택과 파라미터 묶음은 **설정 세트** 로 저장합니다.
로봇 설계	구동 방식(차동 · 전방향 · 차량형), 외곽(footprint), 물리 한계, 센서(라이다 · ToF · 초음파 · Depth)의 종류·위치·각도와 인식 방식을 정합니다. 여기서 정한 값으로 시뮬레이터와 Nav2 설정이 자동으로 만들어집니다.
벤치마크	최대 4개 설정을 같은 시나리오(정상 · 맵에 없는 장애물 · 보행자 · 낮은·공중 장애물 · 단차)로 반복 주행해 성공률, 충돌, 낙하, 속도, 이격, 복구 횟수를 비교합니다.
자동 튜닝	고른 파라미터의 범위 안에서 점수가 가장 높은 조합을 찾습니다. 찾은 값은 바로 적용하거나 다른 맵에서 검증합니다.

필요한 환경

	Docker 로 실행 (권장)	직접 설치
운영체제	Windows 10/11, macOS, Linux	Ubuntu 22.04 (WSL2 포함)
필요 소프트웨어	Docker Desktop	ROS 2 Humble + navigation2, Python 3.10
함께 받을 것	Nav Studio 소스 — `github.com/Hwan-Yeong/nav2_simulator_app` (Git 또는 ZIP 다운로드). 이 PDF 는 설명서일 뿐이라 이것만으로는 실행할 수 없습니다.	
권장 사양	CPU 4코어 이상 · 메모리 8 GB 이상 · 디스크 4 GB. 벤치마크·튜닝은 동시 주행 1개당 CPU 약 0.5~1코어를 씁니다.	
브라우저	Chrome, Edge, Firefox 최신 버전	

> 시뮬레이션은 실제 시간과 같은 속도(1배속)로 돌아갑니다. Nav2 의 제어 루프가 실제 시계 기준으로 동작하기 때문입니다. 많은 주행이 필요할 때는 벤치마크가 여러 시뮬레이션을 동시에 돌립니다.

A. Docker 로 실행하기 권장

1. **Nav Studio 소스를 내려받습니다.** Git 이 없으면 저장소 페이지의 *Code → Download ZIP* 으로 받아 풀어도 됩니다.

```
git clone https://github.com/Hwan-Yeong/nav2_simulator_app.git
```

2. **Docker Desktop 을 설치하고 실행합니다.** 작업 표시줄의 고래 아이콘이 "Engine running" 이 될 때까지 기다립니다.

3. **내려받은 폴더에서 이미지를 만들고 실행합니다.** 인터넷이 필요하고, 처음 한 번은 ROS 2·Nav2 를 내려받아 빌드하느라 10분 안팎 걸립니다. 그다음부터는 몇 초면 켜집니다.

```
cd nav2_simulator_app
docker compose -f docker/compose.yml up -d --build
```

4. **같은 PC 의 브라우저에서 `http://localhost:8765` 를 엽니다.** `localhost` 는 "지금 이 PC" 라는 뜻이라, Nav Studio 를 실행한 PC 에서만 열립니다. 다른 PC 에서 접속하려면 아래 *다른 PC 에서 접속* 을 보세요.

5. **끄기·다시 켜기**

```
docker compose -f docker/compose.yml stop     # 끄기
docker compose -f docker/compose.yml start    # 다시 켜기
docker compose -f docker/compose.yml logs -f  # 서버 로그 보기
```

> 저장한 로봇, 올린 맵, 벤치마크·튜닝 기록은 `navstudio-data` 볼륨에 남아 컨테이너를 지워도 유지됩니다. **다른 PC 에서 접속:** 기본값은 이 PC 에서만 열리게 되어 있습니다. compose.yml 의 `ports` 를 `"8765:8765"` 로 바꿔 다시 올리면(직접 설치는 `--host 0.0.0.0`) 같은 네트워크의 PC 에서 `http://<실행한 PC 의 IP>:8765` 로 접속할 수 있습니다. 로그인 기능이 없으므로 사내망에서만 여세요.

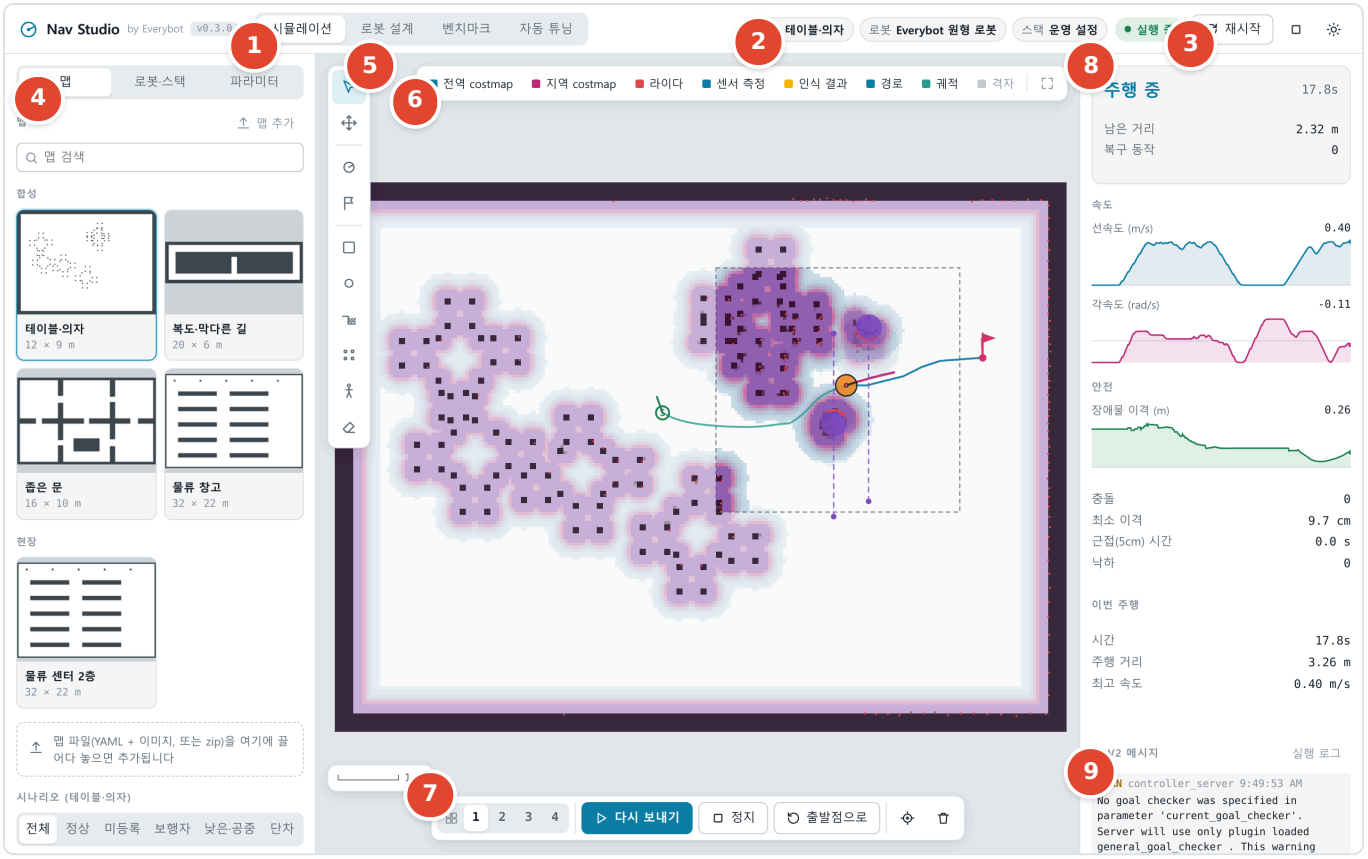
B. Ubuntu 22.04 + ROS 2 Humble 에 직접 설치하기

```
sudo apt install ros-humble-navigation2 python3-venv
git clone https://github.com/Hwan-Yeong/nav2_simulator_app.git && cd nav2_simulator_app
python3 -m venv ~/.navstudio-venv
~/.navstudio-venv/bin/pip install -U pip setuptools
~/.navstudio-venv/bin/pip install ".[tune]"
ln -s ~/.navstudio-venv/bin/navstudio ~/.local/bin/
navstudio build-plugins      # Everybot 플래너·BT 플러그인 빌드 (처음 한 번, 약 1분)
navstudio                    # 서버를 띄우고 브라우저를 엽니다
```

`navstudio doctor`	ROS 2, Nav2, 플러그인, Python 패키지 설치 상태를 점검합니다.
`--port 8765`	웹 주소의 포트. 이미 쓰는 포트면 바꿉니다.
`--host 0.0.0.0`	다른 PC 의 브라우저에서도 접속할 때.
`--domain 91`	시뮬레이션이 쓰는 ROS_DOMAIN_ID. 같은 PC 에서 여러 명이 쓸 때 서로 다르게 정합니다.
`--no-browser`	브라우저를 자동으로 열지 않습니다.

> Nav Studio 는 사용자 셸에 설정된 다른 ROS 작업공간이나 DDS 설정을 쓰지 않습니다. 모든 통신은 이 PC 안(localhost)에서만 오가므로 같은 네트워크의 실제 로봇에 영향을 주지 않습니다.

시뮬레이션 화면

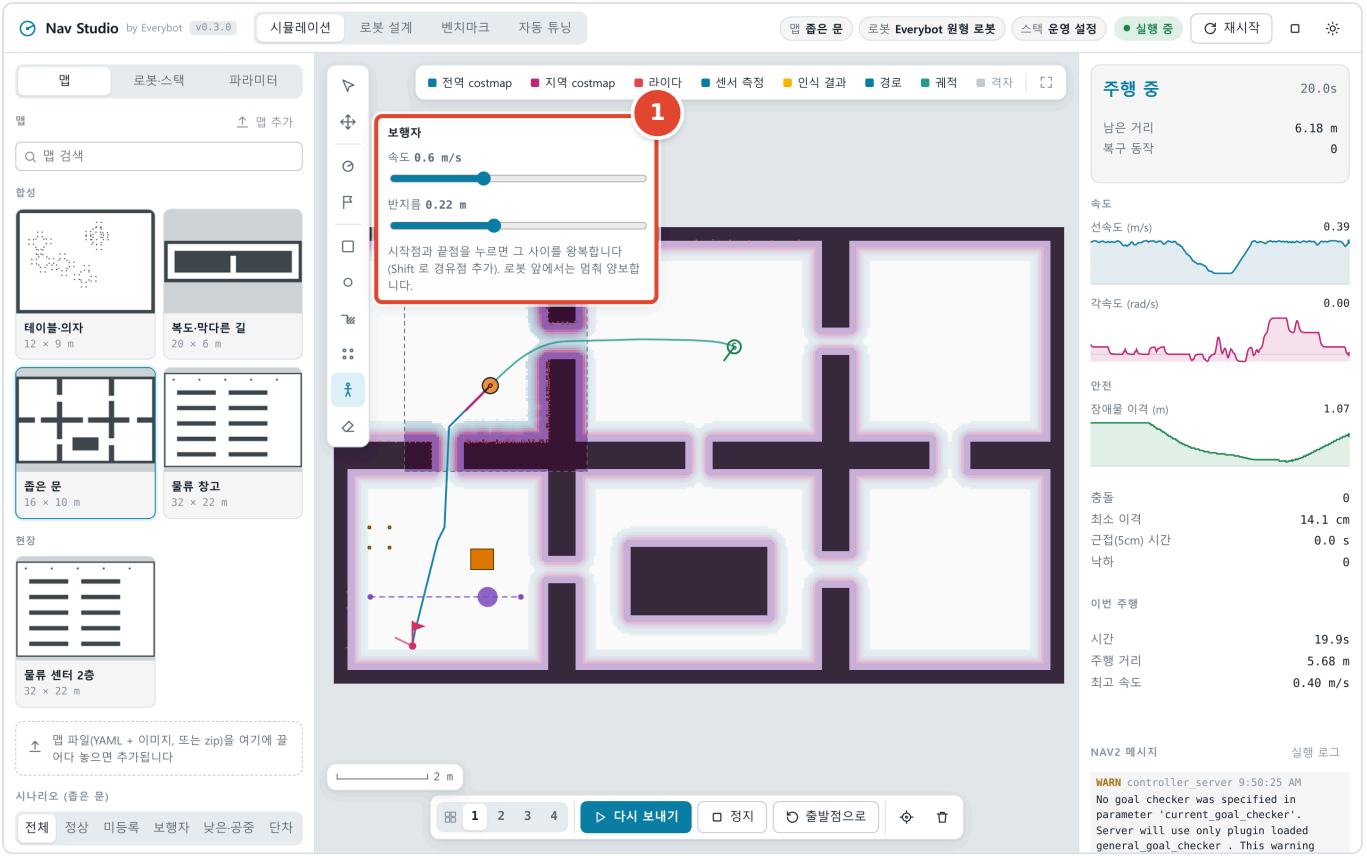


1. **작업 공간 전환** — 시뮬레이션 · 로봇 설계 · 벤치마크 · 자동 튜닝
2. **현재 설정과 상태** — 맵 · 로봇 · 스택(변경 개수), 실행 상태(정지됨 / 시작 중 / 실행 중 / 오류)
3. **시작 · 변경 적용** — 정지 상태면 "시뮬레이션 시작", 설정을 바꾸면 "변경 적용 (재시작)" 으로 바뀝니다
4. **왼쪽 패널** — 맵 고르기·추가·시나리오 불러오기 / 로봇·스택·설정 세트 / 파라미터 조정
5. **편집 도구** — 선택 · 화면 이동 · 출발점 · 목표 · 박스 · 기둥 · 단차 · 의자 · 보행자 · 지우개
6. **레이어** — 전역·지역 costmap, 라이다 점, 센서 측정, 인식 결과, 경로, 지나온 궤적, 격자. 오른쪽 끝 버튼은 맵 전체 보기
7. **주행 막대** — 동시 실행 개수(1~4) · 주행(현재 목표로 다시 보내기) · 정지 · 출발점으로 · 로봇 따라가기 · 장애물 모두 지우기
8. **주행 상태** — 대기 / 주행 중 / 도착 / 실패, 남은 거리, 복구 동작 횟수, 아래에 속도·이격 그래프와 이번 주행 기록
9. **Nav2 메시지** — Nav2 노드의 경고·오류를 실시간으로 보여 줍니다. "실행 로그" 는 전체 출력

> 지도 조작: 휠로 확대·축소, 빈 곳을 끌면 이동합니다. 출발점 **S** 는 초록 원, 목표는 분홍 깃발, 파란 선은 계획 경로, 청록 선은 실제로 지나온 궤적입니다.

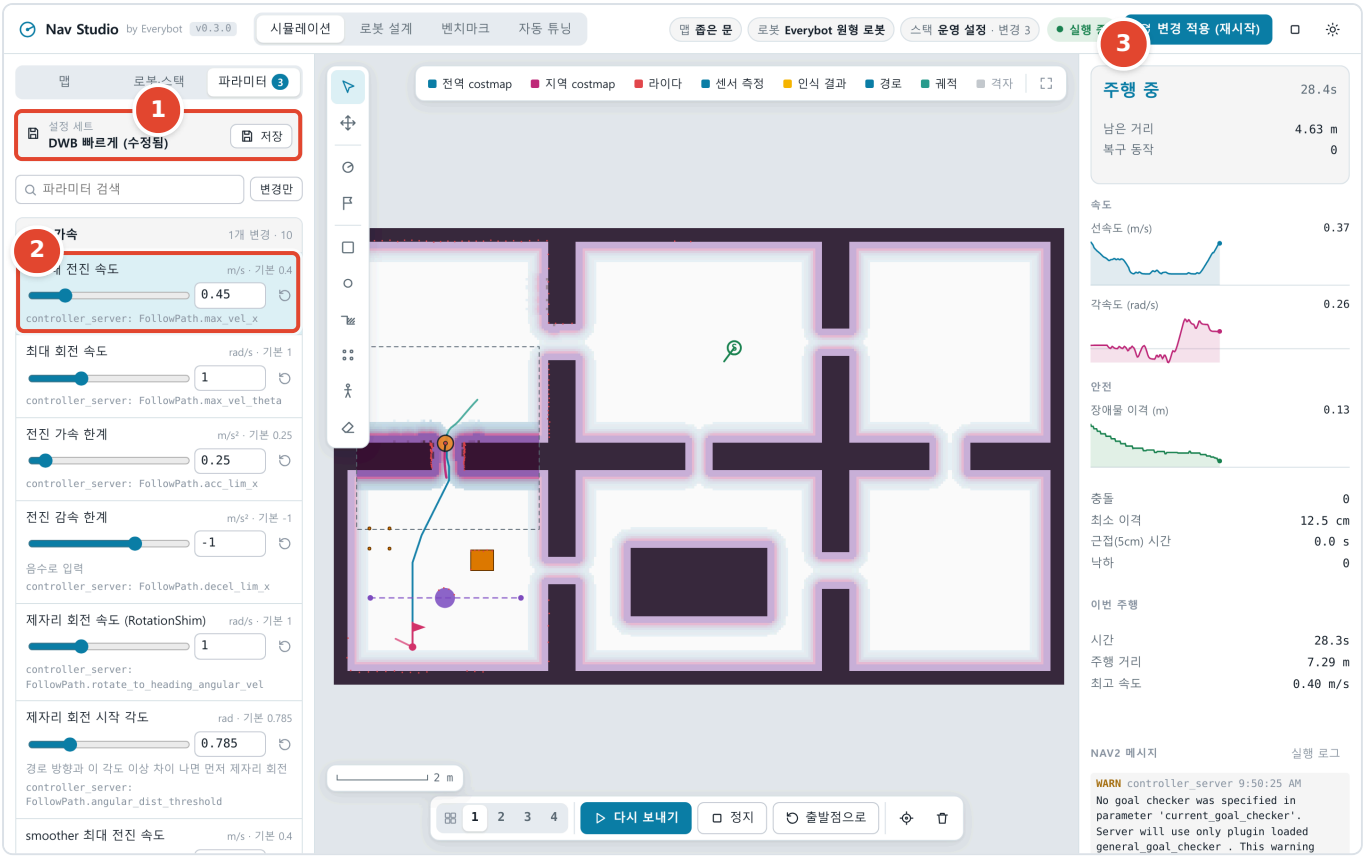
04 시뮬레이션 따라하기

1. **맵 고르기** — 왼쪽 **맵** 탭에서 맵 카드를 누릅니다 (내 맵은 **맵 추가** 로, 12장). 아래 시나리오 목록에서 미리 만든 상황(정상 · 미등록 장애물 · 보행자 · 낮은·공중 장애물 · 단차)을 불러올 수도 있습니다.
2. **시작** — 오른쪽 위 **시뮬레이션 시작**. 약 4초 뒤 "실행 중" 이 됩니다. 차량형 로봇은 약 12초 걸립니다.
3. **출발점** — 출발점 도구(`S`)로 지도를 누른 채 끌면 로봇이 그 자리·방향으로 옮겨집니다.
4. **목표** — 목표 도구(`G`)로 누른 채 끌어 도착 방향을 정하면 바로 출발합니다.
5. **장애물 놓기** — 아래 도구로 주행 중에도 놓을 수 있습니다. 맵에 없는 물체라서, 지도에 없는 짐이나 사람·턱·단차를 만난 상황을 재현합니다.
6. **지켜보기** — 오른쪽 패널의 상태가 "도착" 이 되면 성공입니다. 실패하면 Nav2 메시지에서 이유를 확인합니다.



박스(주황), 의자 다리 4개, 왕복하는 보행자(보라)를 놓은 장면. 보행자 도구를 고르면 속도·크기 옵션이 나타납니다 ①.

도구	키	사용법
선택	`V`	장애물을 끌어 옮기고 `Delete` 로 지웁니다. 빈 곳을 끌면 화면 이동
박스	`B`	끌어서 사각형 장애물을 그립니다. 옵션에서 높이(일반 · 낮은 턱 · 문턱 · 공중, 9장)
기둥	`P`	누르는 곳에 원기둥. 반지름·높이는 옵션에서
단차	`D`	끌어서 바닥이 꺼진 곳(계단·낭떠러지)을 그립니다. 깊이는 옵션에서
의자	`C`	지름 5 cm 다리 4개. 2D 라이다로는 잘 안 보이는 대표적인 장애물
보행자	`W`	시작점·끝점을 누르면 그 사이를 왕복합니다. 로봇 앞에서는 멈춰 양보합니다
지우개	`E`	장애물이나 보행자 경로를 눌러 지웁니다. `Esc` 는 그리던 것 취소



1. **설정 세트** — 지금 쓰는 세트 이름. 값을 바꾸면 "(수정됨)" 이 붙습니다. 저장 버튼으로 이름 붙여 저장합니다
2. **바꾼 항목** — 파란 배경과 점으로 표시됩니다. 오른쪽 되돌리기 버튼으로 기본값 복원. 아래 회색 글씨는 실제 Nav2 파라미터 경로
3. **변경 적용 (재시작)** — 누르면 같은 장면으로 Nav2 를 다시 띄웁니다 (약 4초)

항목 구성

선택한 스택(제어기·경로 계획기)에 맞는 항목만 보입니다: 속도·가속, DWB 평가 가중치, MPPI, Pure Pursuit, Costmap, 경로 계획, 목표·진행 판정. 검색창과 **변경만** 버튼으로 찾습니다.

설정 세트

설정 세트 는 주행 스택과 바꾼 파라미터를 묶은 것입니다. **저장** 을 누르고 이름을 적으면 **로봇·스택** 탭 목록에 들어가고, 동시 실행 슬롯과 벤치마크에서 이름으로 고를 수 있습니다. 같은 이름이면 덮어쓸지 묻습니다. 기본 세트 *튜닝안 A·B* 는 검증 분석에서 나온 값이고 지울 수 없습니다.

고급 오버라이드 · YAML

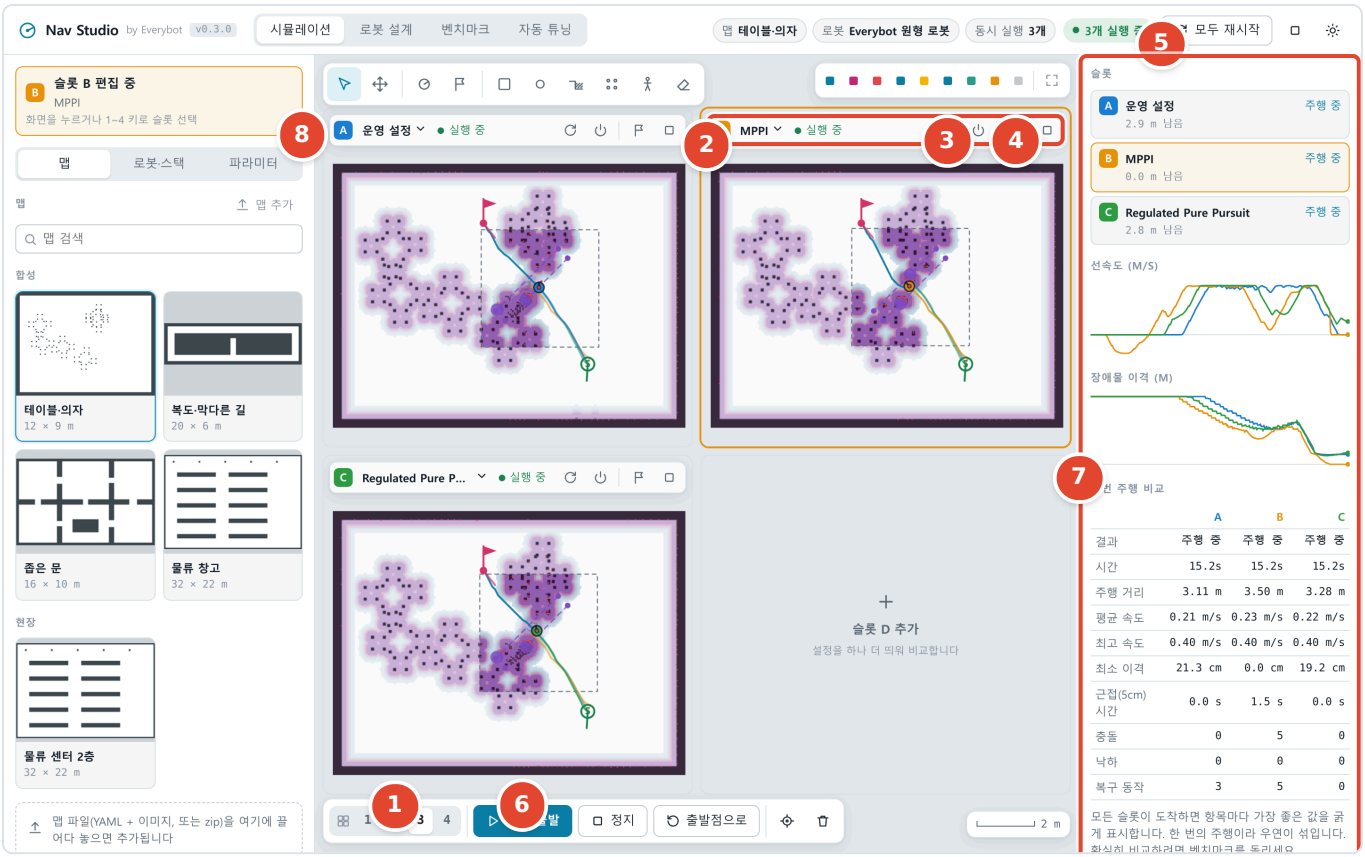
목록에 없는 Nav2 파라미터는 **고급 오버라이드** 에 YAML 로 적습니다. 예: `controller_server: {FollowPath.max_vel_x: 0.3}`. **YAML 미리보기** 는 운영 기본값 대비 바뀐 줄과 최종 Nav2 params 를 보여 주고, 실제 로봇에 쓸 파일로 저장할 수 있습니다.



로봇·스택 탭: 로봇, 주행 스택, 설정 세트 목록. 세트를 누르면 그 스택과 값이 들어오고, 내 세트는 휴지통으로 지웁니다.

06 동시 실행 비교

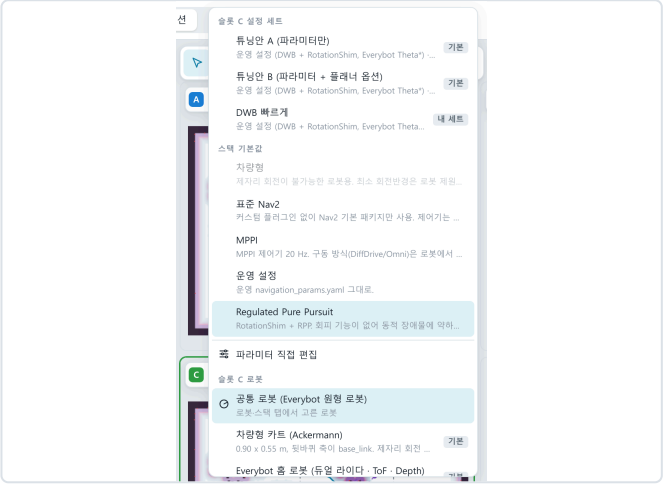
설정 여러 개를 한 화면에서 나란히 돌려 봅니다. 맵과 장면은 모든 슬롯이 같이 쓰고, 슬롯마다 스택과 파라미터가 다릅니다. 센서 배치를 비교하려면 슬롯마다 로봇도 바꿀 수 있습니다.



1. **동시 실행 개수** — 1~4. 슬롯 A–D 는 색으로 구분됩니다. 슬롯 하나가 CPU 약 1코어를 씁니다
2. **슬롯 머리글** — 설정 이름을 누르면 설정 세트·스택 기본값·슬롯 로봇을 고르는 메뉴. 옆은 상태(꺼짐 / 시작 중 / 실행 중 / 변경됨 / 오류)
3. **슬롯 시작 · 끄기** — 그 슬롯만 시작·재시작(설정을 바꿨으면 변경 적용)·끄기
4. **슬롯 주행** — 깃발: 그 슬롯만 출발점에서 다시 주행 · 네모: 그 슬롯 주행 멈춤
5. **모두 시작** — 꺼져 있거나 설정이 바뀐 슬롯을 2초 간격으로 띄웁니다. 모두 실행 중이면 모두 재시작
6. **동시 출발** — 로봇을 모두 출발점에, 보행자를 처음 자리에 되돌린 뒤 같은 순간에 출발. 목표를 찍어도 같습니다
7. **비교 패널** — 슬롯별 상태, 속도·이격 그래프, 이번 주행 비교표. 모두 도착하면 항목마다 가장 좋은 값이 초록
8. **편집 중인 슬롯** — 화면을 누르거나 1~4 키로 고릅니다. 왼쪽 스택·파라미터 탭이 이 슬롯을 바꿉니다

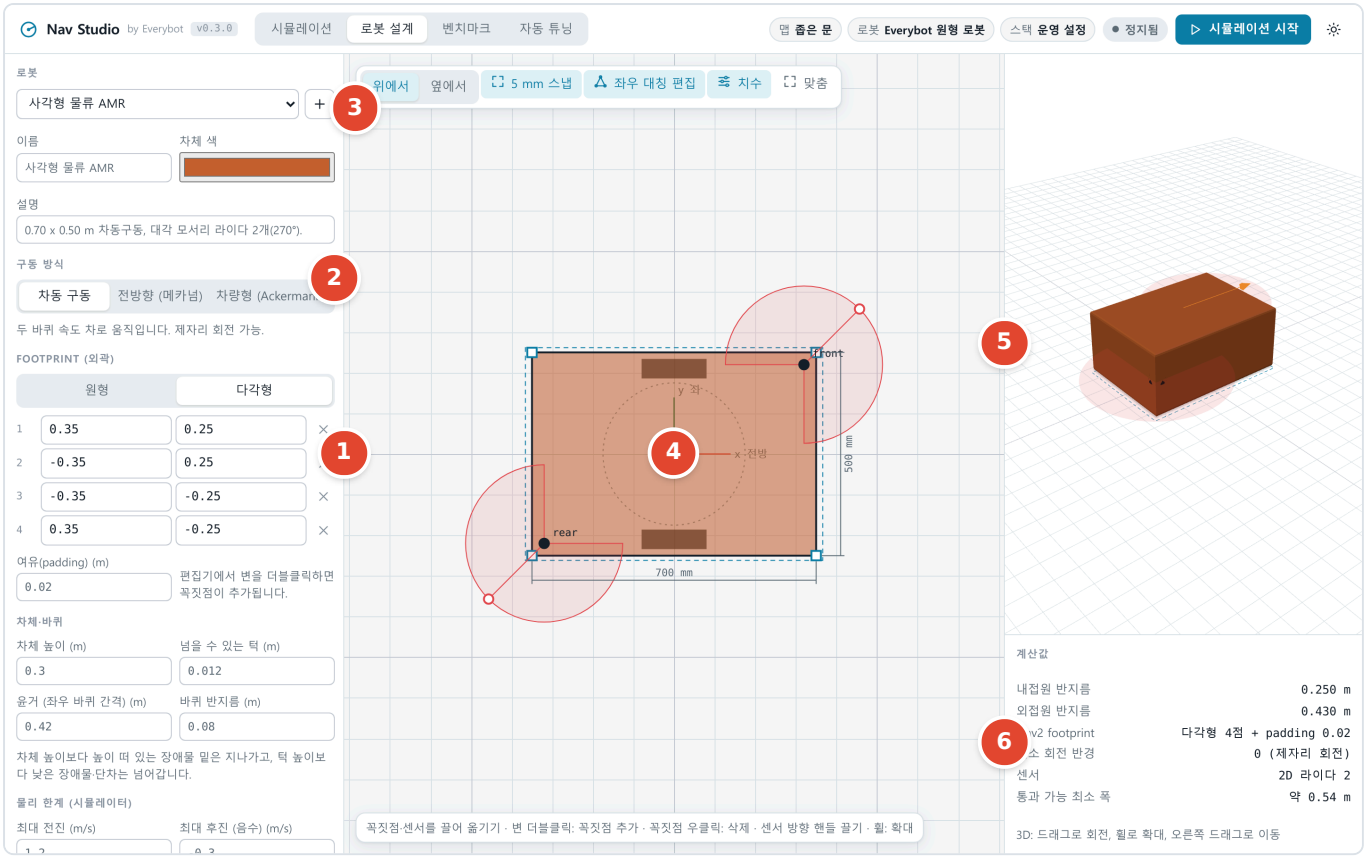
따라하기

1. 맵과 시나리오를 고르고 아래 막대에서 **3** 을 누릅니다.
2. 슬롯 머리글의 이름을 눌러 비교할 설정을 고릅니다. 예: A 운영 설정, B MPPI, C Regulated Pure Pursuit.
3. 상단 **모두 시작**. 세 슬롯이 '실행 중' 이 되면 목표를 찍거나 **동시 출발**.
4. 다른 슬롯의 궤적이 흐린 색으로 겹쳐 보입니다. 한 번의 주행에는 우연이 섞이므로, 차이가 작으면 **이 설정들로 벤치마크** 로 여러 시나리오에서 확인합니다.



슬롯 머리글 메뉴: 설정 세트, 스택 기본값, 다른 슬롯 설정 복사, 파라미터 직접 편집, 이 슬롯만 쓸 로봇.

07 로봇 설계



1 **제원 입력** — 이름, 차체 색·높이, 바퀴, 물리 한계(최대 속도·가속), 구동계 지연·슬립, 라이다 목록

2 **구동 방식** — 차동 구동 · 전방향(메카넘) · 차량형(Ackermann). 차량형은 축거와 최대 조향각으로 최소 회전반경이 정해집니다

3 **편집 옵션** — 5 mm 스냅, 좌우 대칭 편집(꼭짓점 하나를 옮기면 반대쪽도 같이), 치수 표시, 맞춤 보기

4 **상면도 편집기** — 네모 핸들(꼭짓점)을 끌어 외곽을 바꾸고, 변을 더블클릭하면 꼭짓점 추가, 우클릭하면 삭제. 라이다는 점을 끌어 옮기고 부채꼴 끝의 원을 끌어 방향을 돌립니다

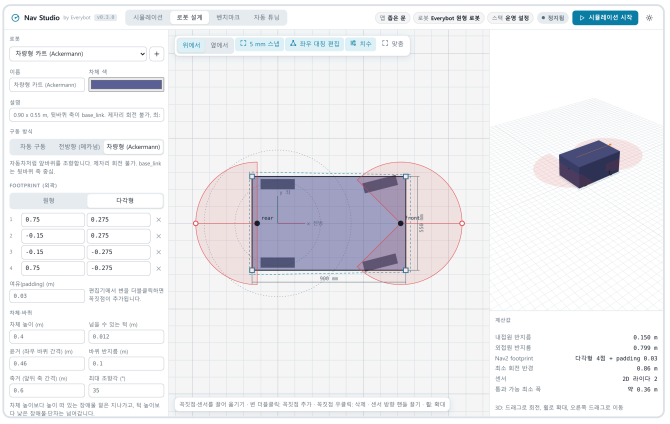
5 **3D 미리보기** — 끌어서 회전, 휠로 확대. 바퀴·라이다·시야 범위를 입체로 확인

6 **계산값** — 내접·외접원, Nav2 footprint, 최소 회전반경, 통과 가능한 최소 폭

구동 방식과 주행 스택

차동 구동	운영 설정(DWB), 표준 Nav2, MPPI, Pure Pursuit
전방향	운영 설정(DWB, 횡이동 포함), 표준 Nav2, MPPI(Omni)
차량형	Smac Hybrid-A* + MPPI(Ackermann). 최소 회전반경은 로봇 제원에서 자동 설정

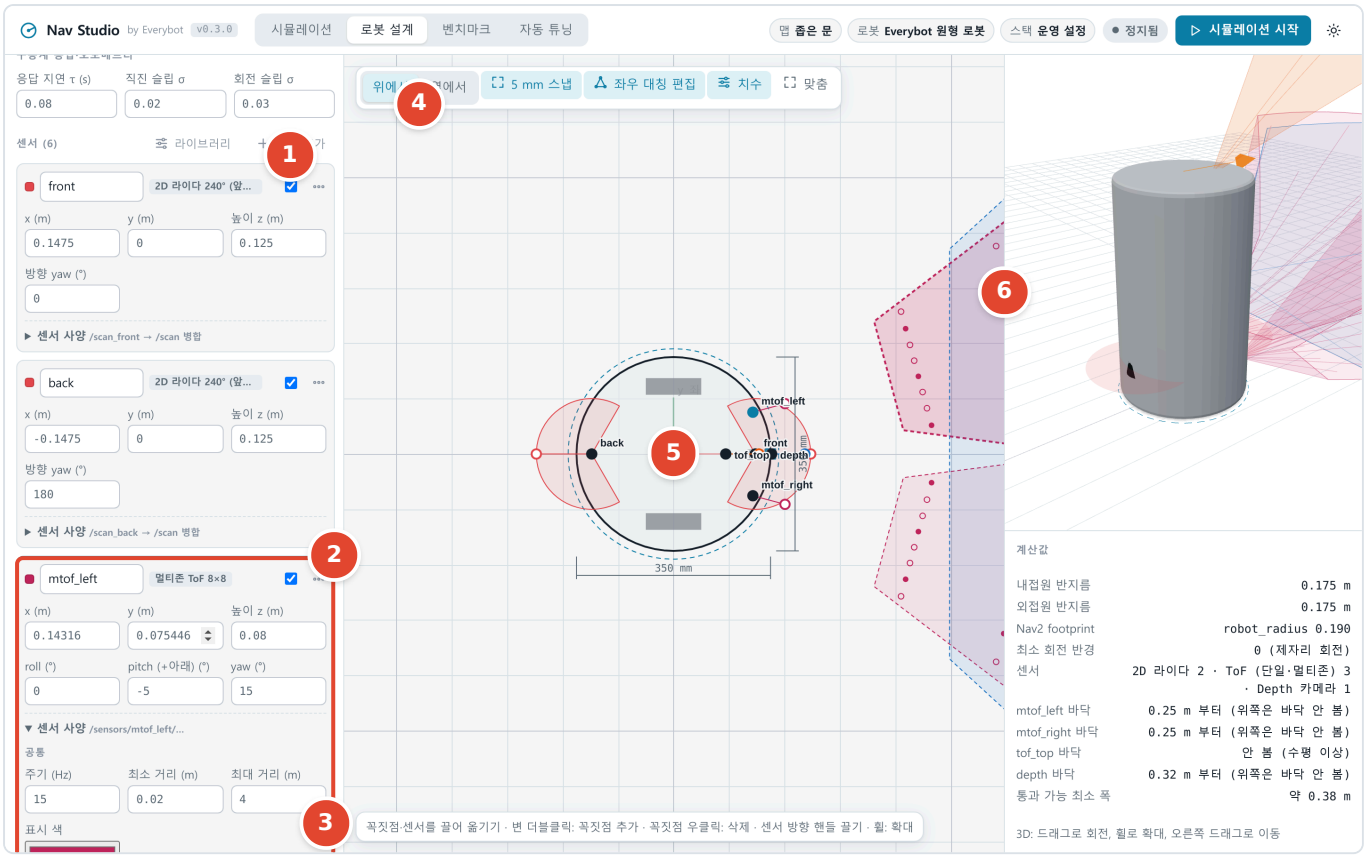
> **저장** 하면 내 로봇 목록에 추가되고, **시뮬레이션에 사용** 을 누르면 바로 시뮬레이션 화면으로 넘어갑니다. 로봇을 바꾸면 그 구동 방식을 지원하는 스택으로 자동으로 바뀝니다. 다각형 로봇은 Nav2 의 inflation 이 외곽을 덮도록 자동으로 맞춰집니다.



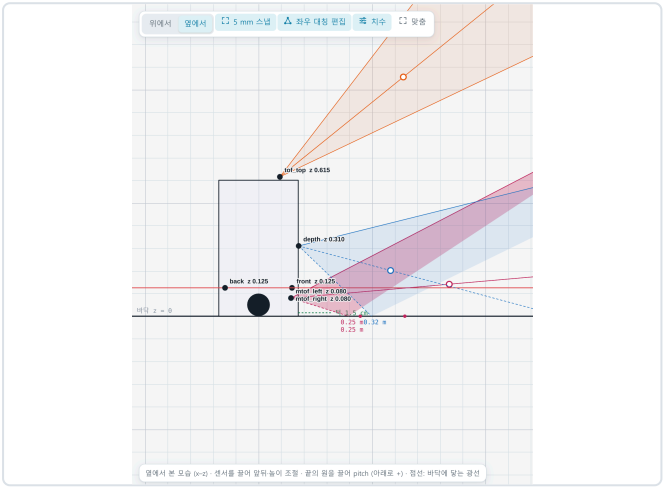
차량형 카트 예: 뒷바퀴 축이 원점, 앞바퀴 조향과 최소 회전반경 원(파란 점선)이 표시됩니다.

08 센서 설계

로봇에 2D 라이다 말고도 ToF(단일 · 멀티존), 초음파, Depth 카메라를 달 수 있습니다. 센서는 **센서 라이브러리** 의 모델(종류 + 사양)을 골라 달고, 로봇 위의 위치와 각도를 정합니다.



1. **라이브러리 · 센서 추가** — 라이브러리에서 센서 모델을 만들고 고칩니다. 센서 추가로 모델을 골라 로봇에 답니다
2. **센서 카드** — 이름, 모델, 켜기/끄기, 위치 x·y·높이 z, 방향 roll·pitch(+ 가 아래)·yaw. ⋯ 메뉴: 복제, 좌우 대칭 복제, 라이브러리 값으로 되돌리기
3. **센서 사양** — 주기·거리·잡음·시야·해상도. 멀티존 ToF 는 존 마스크(쓰는 존만 켜기)
4. **위에서 / 옆에서** — 옆에서 보면 높이와 pitch 를 끌어 맞추고, 아래 광선이 바닥에 닿는 거리가 나옵니다
5. **편집기** — 라이다는 부채꼴, 3D 센서는 시야가 바닥에 닿는 범위(점선)와 존마다 보는 곳(점). 센서를 끌어 옮기고 끝의 원으로 방향을 돌립니다
6. **3D 미리보기** — 센서 상자와 시야 범위, 멀티존 ToF 의 존 광선



옆에서 본 모습: 좌우 멀티존 ToF(아래쪽 광선이 0.25 m 앞 바닥), Depth 카메라, 위쪽 1D ToF, 앞뒤 라이다(가로선).

라이다 병합	라이다가 둘 이상이면 고른 라이다를 360° `/scan` 하나로 합칩니다 (각 라이다는 `/scan_이름` 으로도 나감)
인식 처리	**규칙 기반 인식**(권장): 높이·단차 규칙으로 걸러 여러 프레임 확인 후 costmap 에 올림 · **원시 점**: 점구름을 그대로 · **끔**: 발행만
등반 높이	이보다 낮은 턱은 넘어갑니다. 차체 높이보다 높이 뜬 장애물은 아래로 지나갑니다

> 기본 로봇 **Everybot 홈 로봇** 에 앞뒤 240° 라이다(병합), 좌우 멀티존 ToF 8×8, 위쪽 1D ToF, Depth 카메라가 달려 있습니다. 새 센서 **종류** 는 파이썬 파일 하나로 추가합니다 (라이브러리의 **템플릿 만들기**, 저장소의 `docs/SENSORS.md`).

09 낮은 장애물과 단차

2D 라이다는 자기 높이에 걸리는 것만 봅니다. 5 cm 턱, 테이블 상판, 계단 같은 단차는 바닥을 보는 센서와 인식 처리가 있어야 피합니다.



1. **단차 도구** — 끌어서 바닥이 꺼진 곳을 그립니다 (빗금). 깊이는 옵션에서. 로봇 중심이 들어가면 떨어져 멈춥니다 (낙하)
2. **박스 높이** — 일반(0~1 m) · 낮은 턱(5 cm) · 문턱(1.5 cm) · 공중(0.65~0.75 m) 또는 아래·위 높이 직접 입력. 선택 도구로 고른 장애물도 바꿉니다
3. **레이어** — 센서 측정(센서 색 점, 바닥에 맞은 점은 흐리게) · 인식 결과(확정 장애물 노란 사각형, 단차 ×)
4. **지도** — 5 cm 턱(노란 사각형 = 확정된 인식 결과)이 costmap 에 올라 로봇이 돌아가고, 단차 가장자리에는 × 가 찍힙니다. 아래쪽 공중 장애물(0.70~0.75 m)은 차체보다 높아 그 아래로 지나갑니다
5. **센서 · 인식** — ToF 존 히트맵(센서 뒤에서 본 모습, 빨강 = 가까움), Depth 축소 영상, 1D 거리 막대, 확정 개수, 인식 지우기

인식 규칙 (요약)

- **장애물**: 바닥 위 2 cm ~ 차체+5 cm 에 맞은 점, 위아래 존이 같은 면을 맞힘
- **단차**: 바닥이 보여야 할 존의 거리가 8 cm 이상 길거나 반사가 없음, 바닥보다 3 cm 이상 낮은 점
- 라이다가 이미 본 곳은 빼고, 2 프레임 연속이면 확정. 바닥이 다시 보이면 지움

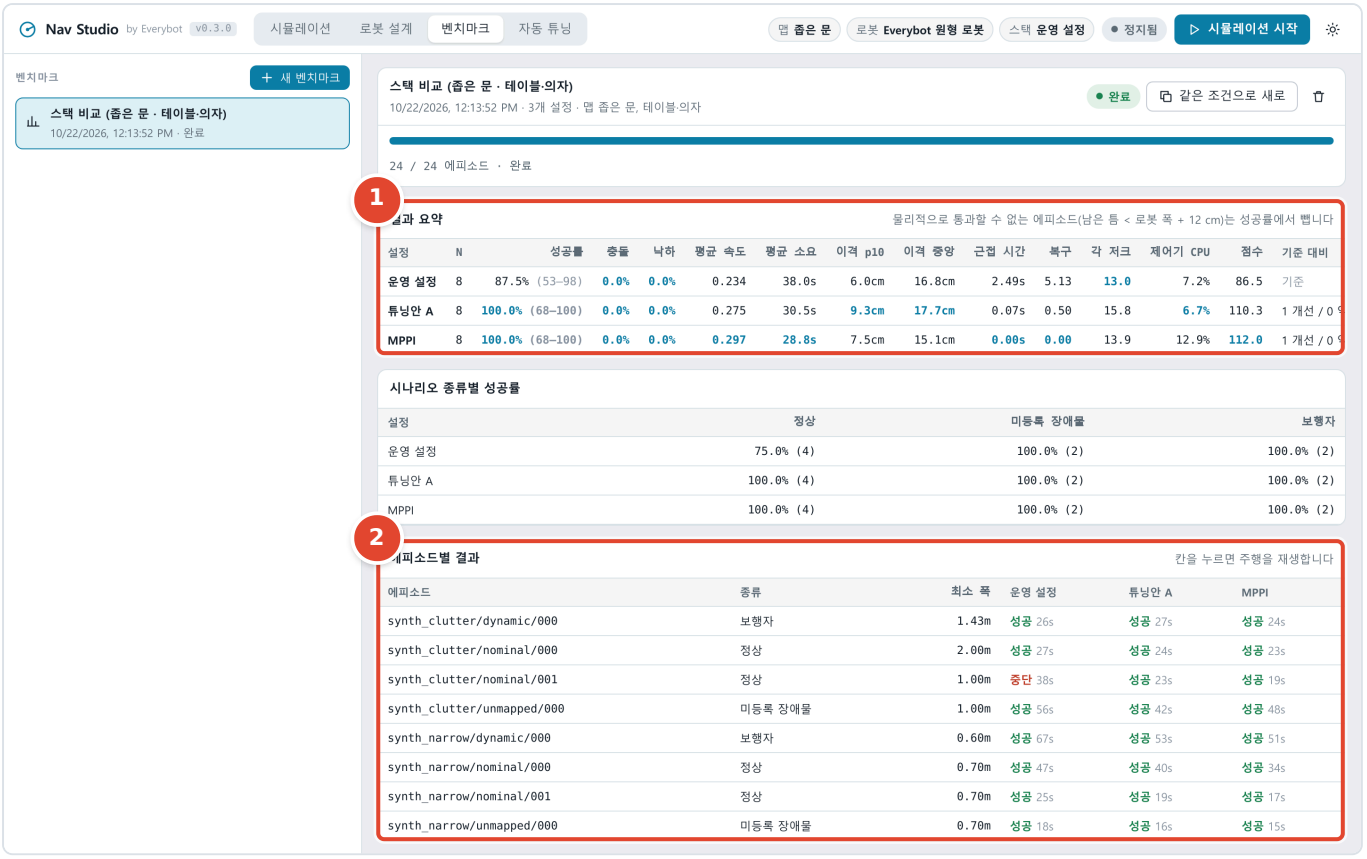
비교와 조정

- 임계값: **파라미터 → 센서 인식** (센서가 있는 로봇에서만 보임)
- 센서 배치 비교: 동시 실행 슬롯 머리글 메뉴의 **슬롯 로봇**
- 벤치마크 시나리오 **낮은·공중 장애물**, **단차** · 낙하는 실패로 셉니다

10 벤치마크

설정 여러 개를 같은 시나리오로 반복 주행해 어느 쪽이 나은지 숫자로 비교합니다. 파라미터를 바꾼 효과가 우연인지 확인할 때 씁니다.

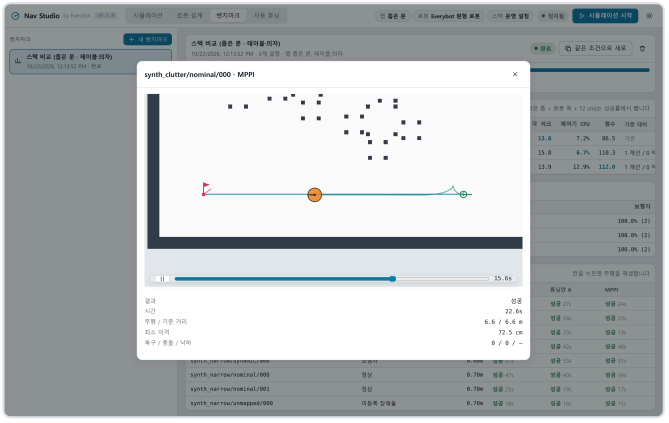
1. **새 벤치마크** → 비교할 설정을 추가합니다 (현재 설정, **설정 세트·스택**, 동시 실행 중인 슬롯 전체). 첫 번째가 비교 기준입니다.
2. 맵, 시나리오 종류, 맵당 에피소드 수, 동시 실행 수를 정합니다. 화면 아래에 예상 시간이 나옵니다.
3. **벤치마크 시작**. 창을 닫아도 서버에서 계속 돌고, 왼쪽 목록에서 다시 열 수 있습니다.



1. **결과 요약** — 설정별 성공률(괄호는 95% 신뢰구간), 충돌, 평균 속도, 소요 시간, 장애물 이격(하위 10%·중앙값), 근접 시간, 복구 횟수, 각 저크, CPU, 점수. 각 열의 가장 좋은 값이 파란색
2. **에피소드별 결과** — 칸을 누르면 그 주행을 재생합니다

결과 읽는 법

- 성공 = Nav2 가 도착을 보고하고 충돌이 없는 주행.
- **기준 대비** 열의 "개선 / 악화" 는 같은 에피소드에서 한 쪽만 성공한 수, p 값이 0.05 보다 작아야 의미 있는 차이입니다.
- 남은 틈이 로봇 폭 + 12 cm 보다 좁은 에피소드는 통과할 수 없으므로 성공률에서 뺍니다.
- "준비된 시나리오" 는 지름 0.35 m 원형 로봇 기준입니다. 큰 로봇은 "로봇 크기에 맞춰 생성" 을 고르세요.



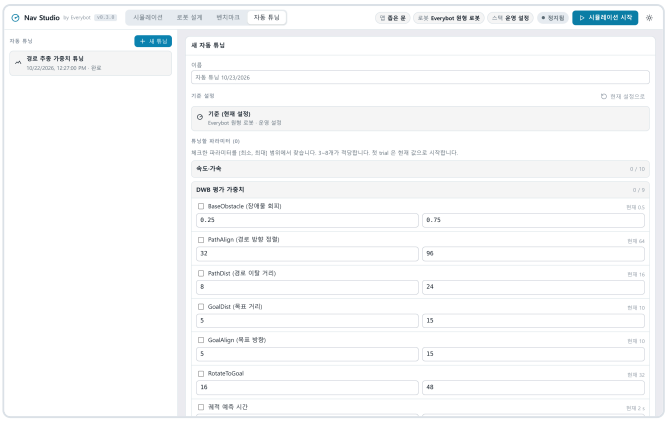
에피소드 재생: 계획 경로, 지나온 궤적, 장애물과 보행자를 시간 순서대로 봅니다.

11 자동 튜닝

고른 파라미터들의 범위 안에서 Optuna(TPE)가 조합을 바꿔 가며 시나리오를 주행하고, 점수가 높은 조합을 찾습니다.

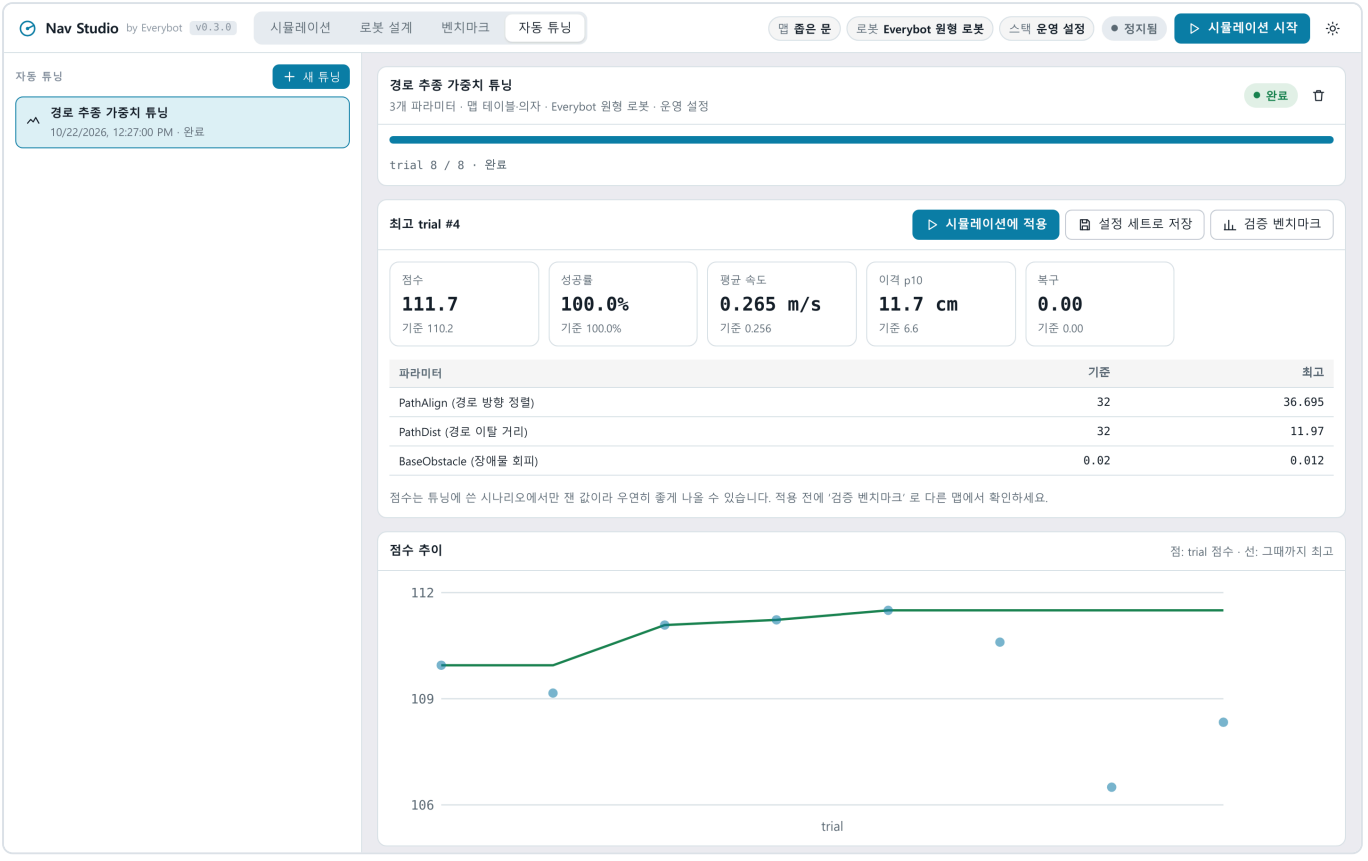
설정 방법

1. **기준 설정**: 지금 시뮬레이션의 로봇·스택·파라미터가 출발점입니다.
2. **튜닝할 파라미터**를 3~8개 체크하고 최소·최대를 정합니다. 첫 trial 은 현재 값으로 시작합니다.
3. **점수 가중치**로 무엇을 우선할지 정합니다: 성공률, 속도, 안전 여유, 부드러움.
4. 맵과 시나리오, trial 수, 동시 trial 수를 정하고 **튜닝 시작**.



새 튜닝 화면: 파라미터 체크와 범위, 점수 가중치, 시나리오, trial 수.

> 점수 = 성공률(충돌 감점) + 평균 속도 + 장애물 이격(하위 10%) − 복구 횟수 − 각 저크. 예상 시간은 trial 하나에 시나리오 수 × 약 45초 ÷ 동시 주행 수입니다.



튜닝 결과: 최고 trial 의 점수와 지표를 기준과 비교하고, 파라미터 값과 점수 추이를 보여 줍니다.

> **적용 전에 검증하세요.** 튜닝 점수는 튜닝에 쓴 시나리오에서만 잰 값이라 우연히 좋게 나올 수 있습니다. **검증 벤치마크** 를 누르면 기준과 최고 설정을 튜닝에 쓰지 않은 맵에서 비교합니다. 결과가 좋으면 **시뮬레이션에 적용** 으로 값을 옮기거나 **설정 세트로 저장** 해 동시 실행·벤치마크에서 씁니다.

기본 맵 4종 외에 ROS map_server 형식의 맵을 추가해 씁니다. 현장 맵처럼 저장소에 넣지 않는 맵도 각자 PC 에 추가하면 됩니다.



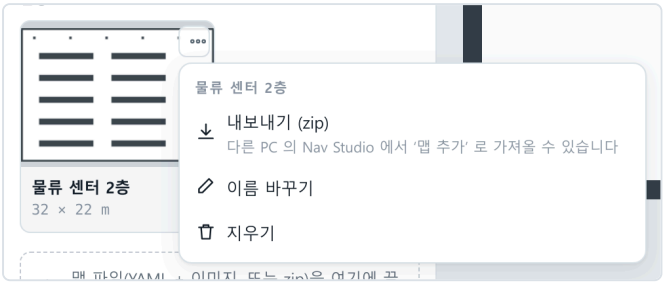
맵 추가 창: 파일을 끌어다 놓거나 눌러서 고릅니다. 맵이 하나면 표시 이름을 정할 수 있습니다.

추가하기

1. 맵 탭의 **맵 추가**. 또는 파일을 맵 탭에 바로 끌어다 놓습니다.
2. 둘 중 하나를 고릅니다: **YAML + 이미지** (resolution · origin · image 가 있는 YAML 과 PGM/PNG), 또는 **맵 zip** (맵 여러 개 가능).
3. **추가** — '내 맵' 에 나타나고 바로 선택됩니다. 같은 이름이 있으면 뒤에 (2) 가 붙습니다.

맵 zip 형식

```
site_a/map.yaml          # ROS map_server YAML
site_a/map.pgm
site_a/info.json         # 선택: 이름·그룹
site_a/scenarios.json    # 선택: 시나리오
site_b/...
```



맵 카드의 ··· 메뉴: 내보내기(zip), 이름 바꾸기, 지우기. 기본 맵은 내보내기만 됩니다.

내보내기 · 이름 바꾸기 · 지우기

맵 카드에 마우스를 올리면 ··· 버튼이 나타납니다. **내보내기** 는 위 형식의 zip 을 받아 다른 PC 의 Nav Studio 에서 그대로 추가할 수 있게 합니다. 지운 맵은 되돌릴 수 없습니다.

명령줄

```
navstudio maps list
navstudio maps import site_maps.zip
navstudio maps import office.yaml office.pgm
--name "3층 사무실"
navstudio maps export site_a -o site_a.zip
navstudio maps remove site_a
```

저장 위치

직접 설치	`~/.navstudio/` — maps(추가한 맵), presets(설정 세트), robots(내 로봇), runs(벤치마크·튜닝 기록), sessions(슬롯별 마지막 세션), overlay(빌드한 플러그인). `NAVSTUDIO_HOME` 으로 위치 변경
Docker	`navstudio-data` 볼륨의 `/data` (위와 같은 폴더 구성)

자주 묻는 것

시작을 눌렀는데 "오류" 가 뜬다	오류 상자의 **로그 보기** 나 오른쪽 아래 **실행 로그** 에서 첫 ERROR 줄을 확인합니다. 고급 오버라이드에 잘못된 값을 넣었을 때 흔합니다. **모두 초기화** 후 다시 시작해 보세요.
"플러그인 없음 → 표준 플래너로 대체" 경고	Everybot 플래너가 빌드되지 않은 상태입니다. `navstudio build-plugins` 를 실행하세요. Docker 이미지는 이미 포함되어 있습니다.
"출발점에서 로봇이 벽과 겹칩니다"	큰 로봇일 때 나옵니다. 출발점 도구로 넓은 곳을 고르세요. 겹친 채로는 경로를 만들지 못합니다.
로봇이 제자리에서 멈춰 복구만 반복한다	좁은 틈 근처에서 장애물 비용이 높은 영역에 들어간 경우입니다. 경로 계획의 통과 비용 가중치를 높이거나 inflation 을 조정해 보세요. 실제 로봇에는 이 상황을 푸는 별도 탈출 동작이 있을 수 있습니다.
동시 실행에서 한 슬롯만 "오류"	그 화면의 **로그 보기** 에서 첫 ERROR 줄을 확인하고 **다시 시작** 합니다. 다른 슬롯은 영향을 받지 않습니다. 기동이 30초 안에 끝나지 않으면 한 번은 자동으로 다시 띄웁니다.
슬롯을 여러 개 띄우니 움직임이 이상하다	슬롯 하나가 CPU 약 1코어를 씁니다. 코어가 부족하면 Nav2 제어 주기가 밀려 결과가 달라집니다. 개수를 줄이거나 벤치마크를 같이 돌리지 마세요.
벤치마크가 느리다	Nav2 는 실시간으로만 돌아서 에피소드당 20~60초가 걸립니다. 동시 실행 수를 CPU 코어 수의 절반 정도까지 올리세요.
낮은 장애물에 부딪히거나 단차에 떨어진다	로봇에 바닥 가까이를 보는 센서(ToF·Depth)가 있고 **인식 처리** 가 '규칙 기반 인식' 인지 확인합니다. 센서를 **옆에서** 보기로 맞춰 아래 광선이 로봇 앞 0.2~0.4 m 바닥에 닿게 하고, 인식 결과 레이어에 점이 찍히는지 봅니다.
포트 8765 를 이미 쓴다	`navstudio --port 8780`, Docker 는 compose.yml 의 `ports` 를 `"127.0.0.1:8780:8765"` 로 바꾸고 `http://localhost:8780` 으로 엽니다.

알아 둘 한계

- 2.5D 시뮬레이션입니다. 장애물은 바닥 모양 + 높이 구간, 단차는 수직으로 꺼진 사각형입니다. 경사로·바닥 재질·RGB 영상·배터리는 다루지 않습니다.
- 위치 추정은 정확하다고 가정합니다. 오도메트리 미끄러짐은 넣었지만 위치 추정 오차는 없습니다.
- 보행자는 정해진 경로를 왕복하는 단순 모델입니다.
- 실제 로봇의 저수준 안전 동작은 포함하지 않습니다. 결과는 설정 간 비교로 읽고, 최종 판단은 실제 로봇 시험으로 합니다.